SLIIT | FACULTY OF COMPUTING | DEPARTMENT OF SOFTWARE ENGINEERING

SE3090 – Software Engineering Frameworks

Lecture 04

Database Design, Authentication, Authorization and Integration

PostgreSQL schema design • JWT authentication • Role-based access control • Secure full-stack integration

Year 3 • Semester 1 • 4 Credits

Lecturer: Eishan Weerasinghe • eishan.w@sliit.lk

Where This Lecture Fits in SE3090

L01	L02	L03*	TODAY	Next
Framework fundamentals & trends	Advanced React frontend	C# / .NET & REST APIs	Database, AuthN/AuthZ & integration	Agentic software development

- **Module content area 2(f)–2(h): database design, authentication & authorization, secure integration**
- Builds on your React frontend (Lecture 02) and ASP.NET Core REST API knowledge
- Directly assessed in lab submissions, the mini hackathon and the Main Assignment (25%)
- Foundation for DevOps deployment topics later (database hosting, environment configuration)

Lecture Learning Outcomes

LO2

Apply suitable frameworks & tools

Design a normalized PostgreSQL schema and integrate it with an ASP.NET Core backend for a full-stack application.

LO3

Use best practices for integration & quality

Implement secure frontend–backend–database integration with validation, safe error handling and secure configuration.

LO4

Select & justify approaches

Justify choices of database design, token-based authentication (JWT) and role-based access control for project requirements.

Why Database Design & Authentication Matter

Data outlives code

Frontends are rewritten every few years; your data and schema live for decades. Poor design today is expensive forever.

Most breaches target data

Broken access control is the #1 risk in the OWASP Top 10. Injection and identification failures also remain in the list.

Performance starts at the schema

No amount of caching rescues a schema with duplicated, inconsistent data and missing indexes.

Every real app has users & roles

E-commerce, banking, LMS, healthcare — all need login, identity and per-role permissions to function legally and safely.

Lecture Roadmap – 3 Hours

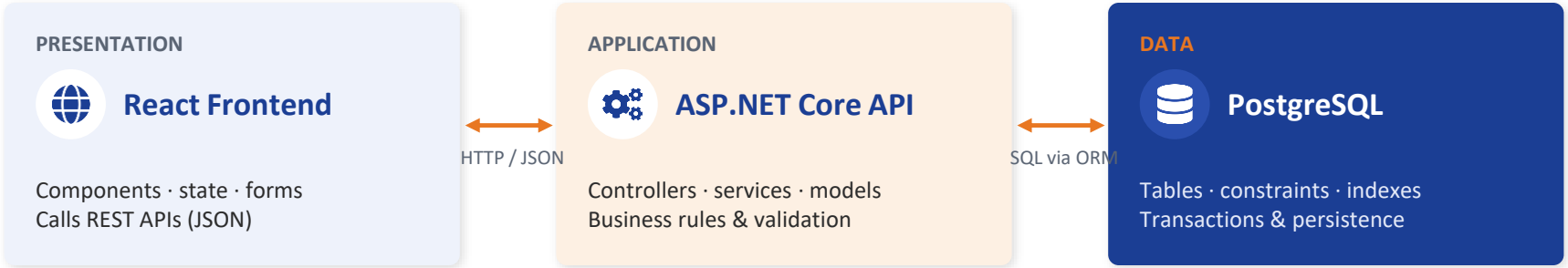
1	**Database design fundamentals** Role of PostgreSQL in full-stack apps · logic vs persistence	~25 min
2	**ER modelling, normalization & schema** Entities · relationships · 1NF–3NF · PostgreSQL schema design	~45 min
3	**Backend ↔ PostgreSQL integration** Connections · env vars · CRUD · repository layer · ORM · pooling	~35 min
4	**Authentication & JWT** AuthN vs AuthZ · sessions vs tokens · JWT flow · token security	~40 min
5	**RBAC & secure integration** Roles & permissions · protected endpoints · validation · safe errors	~35 min

PART 1

Database Design Fundamentals for Full-Stack Applications

- Role of the database in the three-tier architecture
- Relational database concepts refresher
- Why PostgreSQL for modern systems
- Application logic vs data persistence

The Database in Full-Stack Architecture



- **The database is the single source of truth — every tier above it depends on its correctness**
- Stores state that must survive restarts, crashes and deployments (persistence)
- Enforces integrity with constraints even when application code has bugs
- Serves many clients at once: web app, mobile app, reports, background jobs
- The frontend NEVER talks to PostgreSQL directly — always through the API tier

Relational Database Concepts – A Quick Refresher

- **Data is organised into tables (relations) of rows and columns**
  - Row (tuple) = one record, e.g. one customer; Column (attribute) = one fact about it
- **Every table should have a primary key that uniquely identifies each row**
- **Tables are connected through foreign keys — this is the 'relational' part**
- **SQL is the declarative language for defining and querying data**
- **Database transactions follow the ACID principles:**

  Atomicity · Consistency · Isolation · Durability

`customers`

id (PK)	name	email
1	Amal Perera	amal@mail.lk
2	Nadia Silva	nadia@mail.lk

ACID in one line each

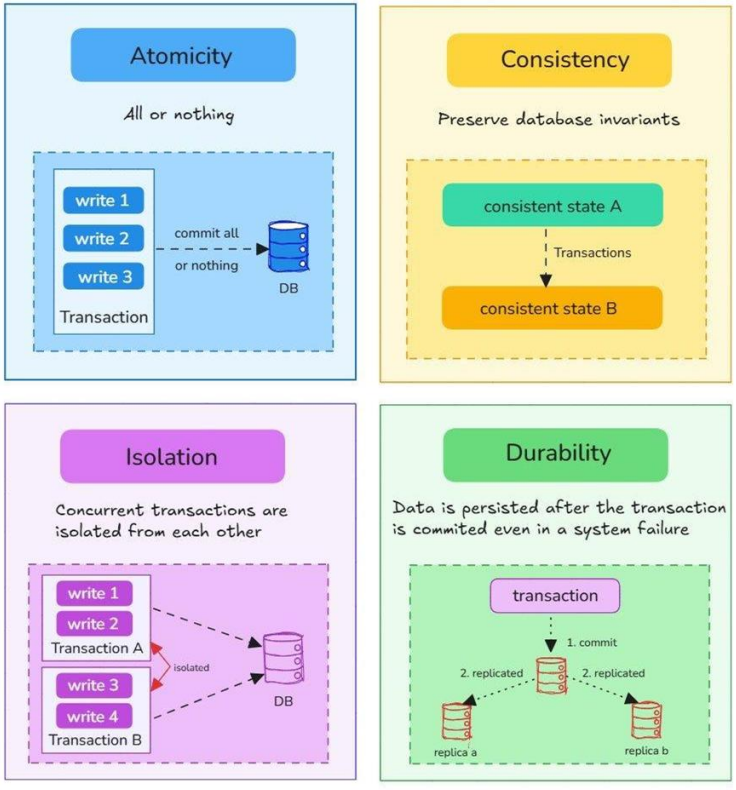
A — the entire operation succeeds or fails.

C — the data always follows the database rules.

I — multiple users can work without interfering with one another.

D — saved data remains available even after a system failure

ACID Principles



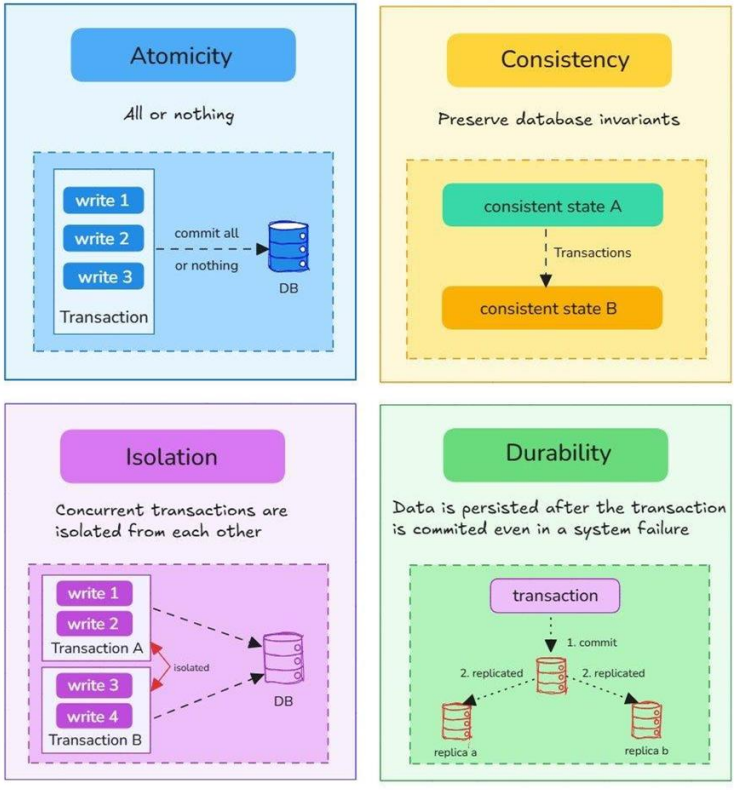
Atomicity — *All or nothing*

- Entire transaction succeeds or fails as a single unit
- If any part fails, database is left completely unchanged
- Prevents partial writes and database corruption

Isolation — *Independent concurrency*

- Concurrent transactions execute without interfering
- Intermediate states are completely hidden from other users
- Provides the illusion of sequential execution

ACID Principles



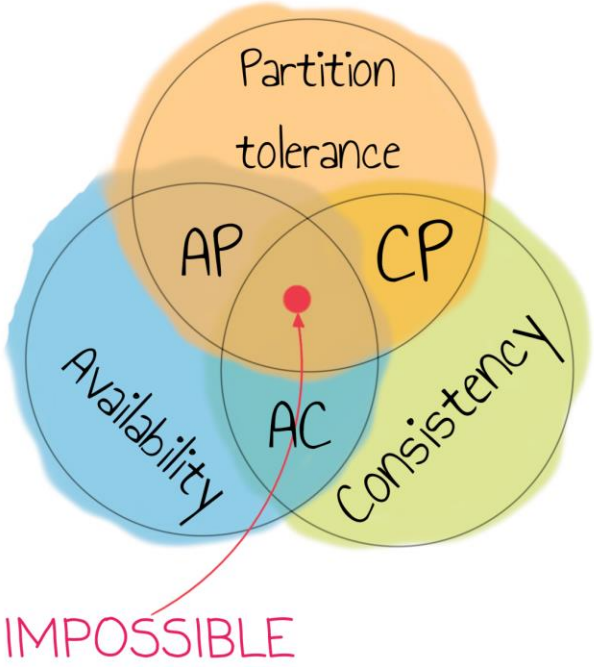
Consistency — *Preserve database invariants*

- Enforces all schema constraints, rules, and triggers
- Database transitions only between valid consistent states
- Prevents illegal data states or foreign key violations

Durability — *Guaranteed persistence*

- Once committed, transaction results are permanent
- Changes survive unexpected power loss or system crashes
- Ensured via write-ahead logging and multi-node replication

CAP Theorem



Consistency (C) — *Absolute correctness*

Every read receives the most recent write, or an error.

Availability (A) — *Non-stop responses*

Every request receives a non-error response, without guaranteeing it contains the most recent write.

Partition Tolerance (P) — *Network resilience*

The system continues to operate despite network delays or dropped connections between nodes.

The Iron Rule: In a distributed system, when a network partition occurs, you must choose between Consistency and Availability.

CAP Theorem

CP Scenario (Consistency + Partition)

- **Prioritizes:** Absolute data correctness and consistency.
- **System Behavior:** Rejects or blocks updates if nodes cannot synchronize, preventing stale reads.
- **Key Tradeoff:** Sacrifices system availability to guarantee data integrity during partitions.
- **Best For:** Banking transactions, financial systems, and billing ledgers.

AP Scenario (Availability + Partition)

- **Prioritizes:** Continuous system uptime and non-stop availability.
- **System Behavior:** Nodes remain fully responsive, returning local data even if it is stale or outdated.
- **Key Tradeoff:** Sacrifices strict consistency; updates propagate eventually once resolved.
- **Best For:** Social media feeds, web caching, and shopping carts.

The Iron Rule: In a distributed system, network partitions are inevitable. You must dynamically choose to prioritize either Consistency (CP) or Availability (AP) during a failure.

Why PostgreSQL for Modern Software Systems?

Open source & free

Permissive licence, no per-core fees; huge community; powers Instagram, Reddit, Skype-scale workloads.

Standards & integrity first

Strong SQL-standard compliance, rich constraints, full ACID transactions, MVCC concurrency.

Rich data types

JSONB for document data, arrays, UUID, full-text search, PostGIS for geo — relational + flexible.

Ecosystem & tooling

First-class support in EF Core (Npgsql), Prisma, Hibernate; runs everywhere — Docker, Azure, AWS RDS, Supabase, Neon.

LO4 link: in your assignment, justify the choice — e.g. "PostgreSQL chosen for ACID guarantees, JSONB flexibility and free cloud hosting tiers."

Application Logic vs Data Persistence

Application logic (API tier)

- Business rules: pricing, discounts, workflows
- Validation of incoming requests
- Orchestration: call services, send emails
- Authentication & authorization decisions
- Changes often — redeployed weekly/daily

Data persistence (DB tier)

- Durable storage of facts (orders, users)
- Structural integrity: keys, constraints
- Transactions & concurrency control
- Efficient retrieval: indexes, query planner
- Changes rarely — schema evolves carefully via migrations

Rule of thumb: the database guarantees data is valid; the application decides what to do with it. Keep business rules out of the schema, and persistence details out of controllers.

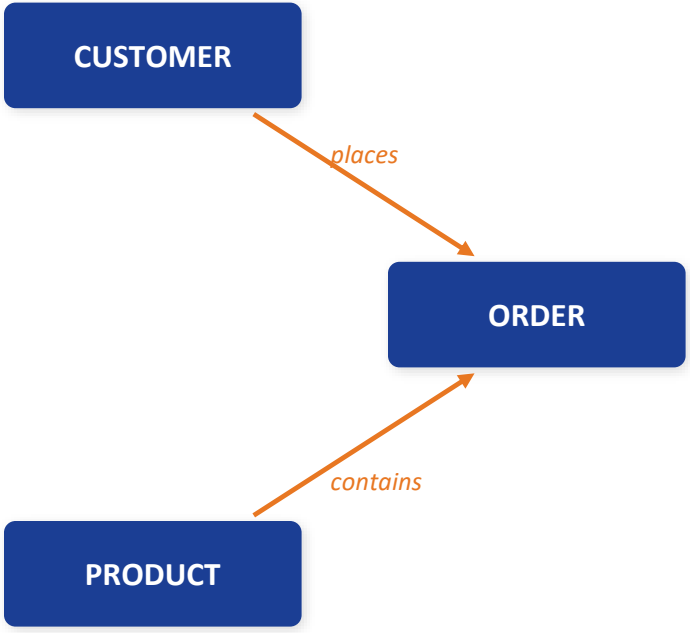
PART 2

Entity Relationships, Normalization & Schema Design

- Entities, attributes and relationship types
- ER diagrams for full-stack applications
- Normalization: 1NF, 2NF, 3NF
- PostgreSQL schema design best practices

Entities, Attributes and Relationships

- **Entity — a real-world 'thing' we store data about**
  - Customer, Product, Order, Category → become tables
- **Attribute — a property of an entity**
  - Customer: id, name, email, phone → become columns
- **Relationship — a meaningful association between entities**
  - A Customer places Orders; an Order contains Products
- **Identify entities from nouns and relationships from verbs in the requirements**



Relationship Cardinalities: 1:1, 1:N, M:N

One-to-one

e.g. each user has one profile



FK with UNIQUE on the child side, or merge into one table.

One-to-many

e.g. one customer, many orders



Most common. FK customer_id **stored** on the **MANY side (orders).**

Many-to-many

e.g. orders ↔ products



Cannot be stored directly — resolve with a **junction table** (order_items).

Key rule: the foreign key always lives on the 'many' side; M:N always becomes a junction (bridge) table.

Primary Keys and Foreign Keys

- **Primary key (PK): uniquely identifies each row — never NULL, never changes**
  - Prefer surrogate keys: id BIGINT GENERATED ALWAYS AS IDENTITY (or UUID)
  - Natural keys (email, NIC) can change — keep them UNIQUE, not primary
- **Foreign key (FK): a column referencing another table's PK**
  - Enforces referential integrity: no order without a real customer
  - ON DELETE behaviour: RESTRICT (default safety) · CASCADE · SET NULL
- **Composite keys: junction tables often use (order_id, product_id) together**

PostgreSQL

```
CREATE TABLE orders (
  id BIGINT GENERATED ALWAYS
     AS IDENTITY PRIMARY KEY,
  customer_id BIGINT NOT NULL
    REFERENCES customers(id)
    ON DELETE RESTRICT,
  status TEXT NOT NULL
    DEFAULT 'PENDING',
  created_at TIMESTAMPTZ
    NOT NULL DEFAULT now()
);
```

Why RESTRICT? Deleting a customer who has orders should fail loudly — losing order history silently (CASCADE) is usually a business disaster.

Surrogate Keys vs Natural Keys

SURROGATE KEY System-generated, meaningless identifier (e.g. Auto-Increment ID, UUID)

When to Use & Advantages

- **Guaranteed Stability:** Never changes, shielding database relationships from shifting real-world data schemas.
- **Performance:** Integer-based keys enable highly optimized indexing, faster joins, and smaller foreign key storage footprint.
- **Universality:** Perfect for entities lacking a clean, immutable natural unique identifier.

Scenario Example

User Accounts: *Storing an internal user ID (e.g.* ***id: 10943****) ensures that if the user updates their registered email address, no foreign key joins cascade or break across order history tables.*

NATURAL KEY Real-world attribute with inherent business meaning (e.g. ISBN, SSN)

When to Use & Advantages

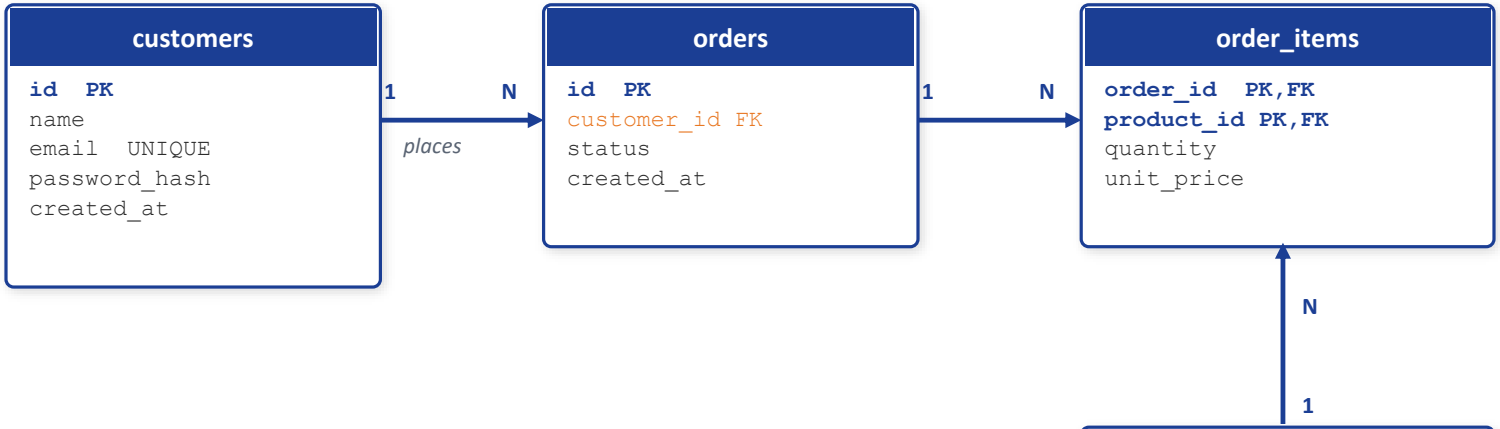
- **Inherent Uniqueness:** Leverages globally unique, strictly regulated standards already guaranteed by business rules.
- **Simpler Queries:** Reduces SQL JOIN density because parent natural key attributes are already directly readable in child tables.
- **Schema Cleanliness:** Avoids creating redundant artificial columns when a completely stable natural ID exists.

Scenario Example

Book Inventory: *Using* ***ISBN-13*** *as the Primary Key. The ISBN of a physical book is globally unique, legally immutable, and standardized worldwide, making an auto-increment ID redundant.*

Core Architectural Rule: When in doubt, prefer **Surrogate Keys** for relational flexibility. If utilizing a Natural Key, ensure the attribute is strictly immutable; if a natural unique attribute is prone to updates (e.g. Email, Username), enforce uniqueness via a **UNIQUE constraint** instead of a Primary Key.

ER Diagram – LankaMart E-Commerce Example



Read the diagram:

- orders↔products was M:N — resolved by order_items
- unit_price copied into order_items: prices change, history must not

Normalization & First Normal Form (1NF)

- **Normalization = organising tables to reduce redundancy and prevent anomalies**
  - Insert, update and delete anomalies all come from storing one fact in many places
- **1NF rules:**
  - Every column holds atomic (indivisible) values — no lists in a cell
  - No repeating groups (phone1, phone2, phone3 …)
  - Each row is unique (has a primary key)
- **Fix: move multi-valued data into its own related table**

✗ **Violates 1NF**

order_id	products
501	Mouse, Keyboard, USB Hub
502	Monitor

✓ **1NF — atomic rows in a child table**

order_id	product	qty
501	Mouse	1
501	Keyboard	1
501	USB Hub	2
502	Monitor	1

Why it matters: try writing SQL to find 'all orders containing a Keyboard' against the comma-separated version — slow LIKE scans, broken counts, impossible joins.

Second and Third Normal Form (2NF & 3NF)

2NF — no partial dependency

- Applies to tables with **composite primary keys**
- Every non-key column must depend on the **entire primary key**, not only part of it

  order_items(order_id, product_id, qty, product_name)
- product_name depends only on product_id, so it should be moved to the ***Products table.***

3NF — no transitive dependency

- Non-key columns must not depend on other non-key columns

  orders(id, customer_id, customer_city)

  – *customer_city* depends on the customer, not the order, so it should be stored in the **Customers table**.

Practical guidance

- 3NF is the standard target for transactional (OLTP) full-stack applications
- Each table should describe exactly ONE entity; each fact stored exactly once
- Denormalize only deliberately, for measured performance reasons — and document why (LO4)

Worked Example: Un-normalized → 3NF

✗ One flat 'orders' spreadsheet-style table

order	customer	email	city	product	cat	price	qty
501	Amal	amal@m.lk	Galle	Mouse	IT	2500	1
501	Amal	amal@m.lk	Galle	Keyboard	IT	5500	1
502	Nadia	nadia@m.lk	Kandy	Mouse	IT	2500	2

Redundancy everywhere: Amal's details ×2, Mouse price ×2 → update/insert/delete anomalies

✓ Decomposed to 3NF (our LankaMart schema)

`customers`

id	name	city
1	Amal	Galle
2	Nadia	Kandy

`products`

id	name	price
10	Mouse	2500
11	Keyboard	5500

`orders`

id	cust_id
501	1
502	2

`order_items`

ord	prod	qty
501	10	1
501	11	1
502	10	2

Each fact lives exactly once; JOINs reassemble the original view when needed.

Normalize or Not?

Trade-offs of Schema Normalization

▲ Advantages (Why we normalize):

- **Eliminates redundancy:** Saves storage and keeps each fact in exactly one place.
- **Data Integrity:** Avoids update, insert, and delete anomalies completely.
- **Simple Updates:** Writing or updating transactional data is extremely fast and consistent.

▼ Disadvantages (The cost):

- **Performance:** Requires complex SQL JOINs to reassemble the original entity view.
- **Overhead:** More tables and relationships mean higher architectural complexity.

Scenarios: When to Choose What

✓ When to Normalize (OLTP Target):

- **Transactional Applications:** Standard target for transactional, full-stack database engines.
- **Frequent Writes:** Systems with highly frequent updates where inconsistencies cannot be tolerated.
- **Strict Consistency:** Financial systems, user profiles, or core operational data stores.

✕ When to Denormalize (OLAP/Scale):

- **Data Warehousing & Dashboards:** Read-heavy analytics where query performance dominates.
- **High-Scale Read APIs:** To eliminate heavy JOIN bottlenecks, do so deliberately and measure results.

PostgreSQL Schema Design Best Practices

Naming & types

- snake_case, plural table names (orders)
- TEXT over VARCHAR(n) unless limits matter
- NUMERIC for money — never FLOAT
- TIMESTAMPTZ (UTC) for all timestamps

Constraints = free correctness

- NOT NULL by default; allow NULL deliberately
- UNIQUE for emails, usernames, codes
- CHECK (price >= 0), CHECK (qty > 0)
- FKs on every relationship

Indexes

- PK & UNIQUE are indexed automatically
- Index FK columns and frequent WHERE/JOIN columns
- Don't over-index — each index slows writes
- EXPLAIN ANALYZE to verify usage

Evolution & safety

- Schema lives in versioned migrations (EF Core)
- Never edit production tables by hand
- Seed data scripts for dev/test
- One schema diagram in your repo & report

Example: LankaMart Core Schema in PostgreSQL

schema.sql

```
CREATE TABLE customers (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now());

CREATE TABLE products (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name TEXT NOT NULL,
  price NUMERIC(10,2) NOT NULL CHECK (price >= 0),
  stock_qty INT NOT NULL DEFAULT 0 CHECK (stock_qty >= 0));

CREATE TABLE order_items (
  order_id BIGINT REFERENCES orders(id) ON DELETE CASCADE,
  product_id BIGINT REFERENCES products(id),
  quantity INT NOT NULL CHECK (quantity > 0),
  unit_price NUMERIC(10,2) NOT NULL,
  PRIMARY KEY (order_id, product_id));

CREATE INDEX idx_orders_customer ON orders(customer_id);
```

Spot the practices

- Surrogate IDENTITY keys
- UNIQUE email (natural key kept unique, not primary)
- password_hash — never plain text
- NUMERIC for money + CHECK guards
- Composite PK on the junction table
- CASCADE only where children are meaningless alone
- Explicit index on the FK used by every 'my orders' query

Checkpoint 01 – Quick Questions

Answer with your neighbour — 4 minutes

Q1 A table stores course codes as "SE3090; IT2030; SE3050" in one column. Which normal form is violated and how do you fix it?

Q2 students(id, module_id, module_name, grade) with key (id, module_id) — which normal form does module_name break?

Q3 Why does order_items store unit_price when products already has price? Is that redundancy?

Q4 Which side of a one-to-many relationship holds the foreign key — and why can't it be the other side?

PART 3

Backend & Database Integration with PostgreSQL



- Connection configuration & environment variables
- CRUD workflow through layers
- Repository / service pattern & ORM (EF Core + Npgsql)
- Error handling & connection pooling

Connecting the API to PostgreSQL — Securely

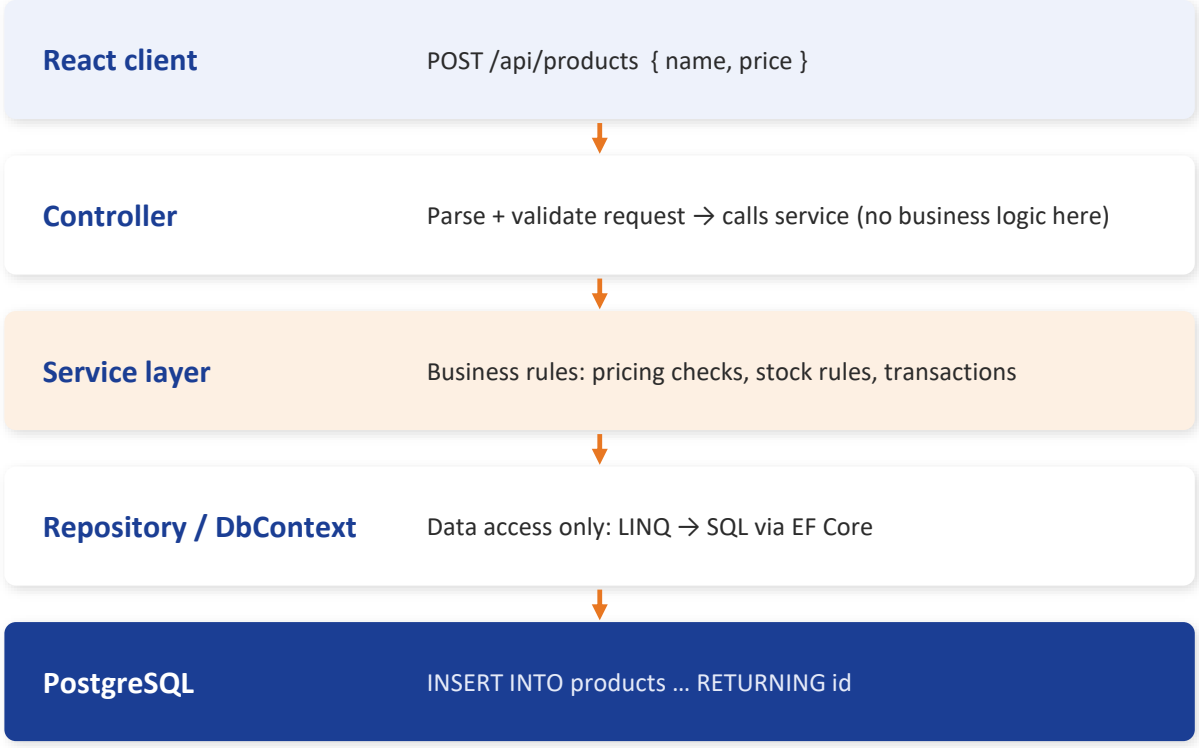
- **A connection string tells the driver where & how to connect**
  - Host, port (5432), database, username, password, SSL mode
- **NEVER hard-code credentials in source code or commit them to Git**
- **Use environment variables / user-secrets per environment**
  - Dev, test and production each get their own values
  - Same code, different config — a 12-Factor App principle
- **Production: secrets come from the platform (Azure App Settings, AWS Secrets Manager)**

```
# .env  (git-ignored!)
DB_CONNECTION="Host=localhost;Port=5432;
  Database=lankamart;Username=app_user;
  Password=${DB_PASSWORD};SslMode=Require"
```

```
// Program.cs
var cs = builder.Configuration
    .GetConnectionString("Default");
builder.Services.AddDbContext<AppDbContext>(
    o => o.UseNpgsql(cs));
```

Checklist: .env in .gitignore · least-privilege DB user (no superuser!) · SSL required · different passwords per environment

CRUD Workflow — Repository / Service Layers



Why layers?

- Separation of concerns — each layer has one job
- Testability: mock the repository, unit-test the service
- Swap implementations (e.g. cache layer) without touching controllers
- CRUD = Create, Read, Update, Delete ↔ INSERT, SELECT, UPDATE, DELETE ↔ POST, GET, PUT, DELETE

ORM Concepts — Entity Framework Core + Npgsql

What is an ORM?

An **Object-Relational Mapper (ORM)** allows a C# application to communicate with a database using **classes and objects**.

In **Entity Framework Core**:

- **classes ↔ tables,**
- **properties ↔ columns,**
- **objects ↔ rows**

- We use **LINQ queries** to retrieve, filter, and sort data.
- We can write queries using LINQ, and **Entity Framework translates** them into parameterized SQL, which helps prevent SQL injection.

EF Core (Npgsql provider)

```
// Entity class ↔ table
public class Product {
  public long Id { get; set; }
  public string Name { get; set; }
  public decimal Price { get; set; }
}

// LINQ → parameterized SQL
var cheap = await db.Products
  .Where(p => p.Price < limit)
  .OrderBy(p => p.Name)
  .ToListAsync();

-- SELECT * FROM products
--  WHERE price < $1 ORDER BY name
```

ORM Concepts — Entity Framework Core + Npgsql

EF Core Features

- **Change tracking** detects modifications to objects, and SaveChanges() updates the database.
- **Migrations** help us manage database schema changes through code.

- **Npgsql** is the PostgreSQL database provider that allows Entity Framework Core to communicate specifically with PostgreSQL.

- **Benefits: productivity, type safety, built-in SQL-injection protection**
- **Costs & cautions:**
  - N+1 query problem — use .Include() deliberately
  - We must still understand SQL and check the generated queries. Problems such as the **N+1 query issue** can reduce application performance

EF Core (Npgsql provider)

```
// Entity class ↔ table
public class Product {
  public long Id { get; set; }
  public string Name { get; set; }
  public decimal Price { get; set; }
}

// LINQ → parameterized SQL
var cheap = await db.Products
  .Where(p => p.Price < limit)
  .OrderBy(p => p.Name)
  .ToListAsync();

-- SELECT * FROM products
--  WHERE price < $1 ORDER BY name
```

Configuring the Model: Constraints

Data Annotations vs Fluent API

EF Core provides two primary ways to configure constraints (Max Length, Required, Column Names, Default values):

- **Data Annotations:** Attributes applied directly to the entity classes. Quick and readable, but mixes domain logic with DB config.
- **Fluent API:** Configured inside the DbContext's OnModelCreating method. Keeps entity classes pure (POCOs) and offers more advanced configuration options not available via attributes.

Best Practice: Use Fluent API configurations separated into individual **`IEntityTypeConfiguration<T>`** classes for clean architecture.

Model Configuration Examples

```
// 1. Data Annotations
public class Product {
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Name { get; set; }
}

// 2. Fluent API (in DbContext)
protected override void OnModelCreating(
ModelBuilder builder) {
    builder.Entity<Product>()
    .Property(p => p.Name)
    .IsRequired()
    .HasMaxLength(100);
}

// 3. Usage
using var db = new AppDbContext();
db.Products.Add(new Product { Name = "Laptop" });
await db.SaveChangesAsync();
```

Database Performance: Creating Indexes

Why use Indexes?

Indexes are special lookup tables that the database search engine can use to speed up data retrieval. Without an index, the DB performs a "Full Table Scan".

- **Primary Keys** are automatically indexed by EF Core.
- **Foreign Keys** are usually indexed automatically, depending on the provider.
- **Custom Indexes:** Should be added to columns heavily used in WHERE, ORDER BY, or JOIN clauses (e.g., Emails, Usernames, Dates).
- **Composite Indexes:** Indexes spanning multiple columns, useful for frequent multi-column queries.
- **Unique Indexes:** Enforces data integrity (e.g., ensuring no duplicate emails).

Creating Indexes (Fluent API)

```
// Creating an Index using Fluent API
protected override void OnModelCreating(
   ModelBuilder mb) {
   // Simple Index
   mb.Entity<User>()
      .HasIndex(u => u.Email);
   // Unique Index
   mb.Entity<User>()
      .HasIndex(u => u.Username)
      .IsUnique();
   // Composite Index
   mb.Entity<Person>()
      .HasIndex(p => new {
         p.FirstName, p.LastName
      });
   // Index using Data Annotation (EF 5.0+)
   // [Index(nameof(Email), IsUnique = true)]
}
```

Relationships: One-to-One

Defining the Relationship

A One-to-One relationship exists when exactly one row in Table A corresponds to exactly one row in Table B. Often used to split wide tables or isolate sensitive data.

- **Navigation Properties:** Both entities contain a reference property to the other entity.
- **Foreign Key:** EF Core requires a Foreign Key on one side of the relationship to determine the dependent entity.
- **Dependent Entity:** In the example, UserProfile is the dependent entity because it holds the UserId FK.

Entity Models (One-to-One)

```
public class User {
   public int Id { get; set; }
   public string Username { get; set; }
   // Reference Navigation Property
   public UserProfile Profile { get; set; }
}

public class UserProfile {
   public int Id { get; set; }
   public string Bio { get; set; }
   // Foreign Key
   public int UserId { get; set; }
   // Reference Navigation Property
   public User User { get; set; }
}

// 3. Usage
using var db = new AppDbContext();
db.Users.Add(new User {
   Username = "alice",
   Profile = new UserProfile { Bio = "Developer" }
});
await db.SaveChangesAsync();
```

Relationships: One-to-Many

The Most Common Pattern

A One-to-Many relationship exists when one row in Table A can be associated with many rows in Table B. (e.g., A Blog has many Posts).

- **Principal Entity:** Contains a Collection navigation property (e.g., ICollection<Post>).
- **Dependent Entity:** Contains a Reference navigation property back to the principal, and a Foreign Key property.
- **Convention:** EF Core automatically discovers this relationship by convention if the naming patterns match.

Entity Models (One-to-Many)

```
public class Blog {
  public int BlogId { get; set; }
  public string Url { get; set; }
  // Collection Navigation Property
  public ICollection<Post> Posts { get; } = new
  List<Post>();
}

public class Post {
  public int PostId { get; set; }
  public string Title { get; set; }
  // Foreign Key
  public int BlogId { get; set; }
  // Reference Navigation Property
  public Blog Blog { get; set; }
}

// 3. Usage
using var db = new AppDbContext();
var blog = new Blog { Url = "dotnet.com" };
blog.Posts.Add(new Post { Title = "EF Core" });
db.Blogs.Add(blog);
await db.SaveChangesAsync();
```

Relationships: Many-to-Many

Implicit Join Tables (EF Core 5.0+)

A Many-to-Many relationship exists when multiple rows in Table A map to multiple rows in Table B. (e.g., Posts and Tags).

- **Modern EF Core:** Handles Many-to-Many implicitly. You only need to define Collection navigation properties on both sides.
- **Join Table:** EF Core automatically generates the intermediate "Join Table" in the database under the hood during migrations.
- **Extra Payload:** If you need to store extra payload data (like a "DateAdded" field) in the join table, you must manually define the join entity class.

Entity Models (Many-to-Many)

```
public class Post {
   public int Id { get; set; }
   public string Title { get; set; }
   // Collection Navigation
   public ICollection<Tag> Tags { get; set; }
}

public class Tag {
   public int Id { get; set; }
   public string Name { get; set; }
   // Collection Navigation
   public ICollection<Post> Posts { get; set; }
}

// EF Core automatically creates a PostTag
// join table with PostId and TagId.

// 3. Usage
using var db = new AppDbContext();
var post = new Post { Title = "Many-to-Many" };
post.Tags = new List<Tag> { new Tag { Name = "EF
Core" } };
db.Posts.Add(post);
await db.SaveChangesAsync();
```

Database Errors & Connection Pooling

Handling database errors

- Database operations may fail due to **timeouts, duplicate values, foreign-key violations, or connection issues.**
- We should catch the **specific database error** and return a meaningful API response. *For example*, PostgreSQL error **23505** means a unique value already exists. The API can return **409 Conflict** with a safe message such as 'Email already registered.
- We should retry only **temporary connection failures**. We must not automatically retry write operations because this could create duplicate records.
- The complete error should be logged on the **server**, but technical or sensitive details should not be shown to the client.

Connection pooling

- A **connection pool** keeps a set of database connections ready for reuse. This improves performance because creating a new connection for every request is expensive.
- **Npgsql manages connection pooling automatically**. When we close a connection, it normally returns to the pool for another request.
- Finally, always dispose of database contexts and connections correctly. Otherwise, connection leaks can exhaust the pool and prevent new requests from accessing the database

PART 4

Authentication, Authorization & JWT

- Authentication vs authorization
- Public vs protected routes; sessions vs tokens
- JWT structure, login flow, access & refresh tokens
- Token storage and common JWT security risks

Authentication vs Authorization

Authentication (AuthN)

"Who are you?"

- Verifying identity: login with email + password
- Also: OAuth (Google login), MFA, biometrics
- Happens first, typically once per session
- Failure → 401 Unauthorized

Authorization (AuthZ)

"What may you do?"

- Checking permissions for an action/resource
- Roles (Admin, Staff, Customer) & policies
- Happens on every protected request
- Failure → 403 Forbidden

Airport analogy: passport control checks WHO you are (authentication); your boarding pass decides WHICH flight and seat you may board (authorization). Both are required — in order.

Protecting Routes: Sessions vs Token-Based Auth

Public vs protected routes

Public: GET /products · POST /auth/register · POST /auth/login
Protected: GET /orders/my · POST /orders · admin endpoints — **require proof of identity on EVERY request**

HTTP is stateless

The server forgets you between requests. After login, every later request must carry proof of identity. Two strategies exist for that proof →

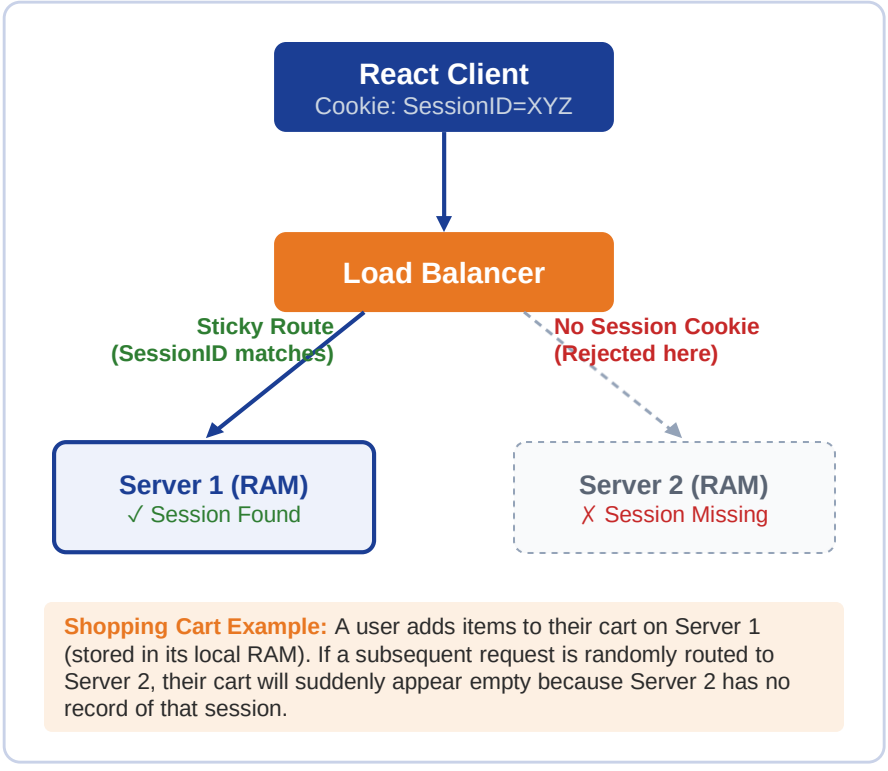
	Server-side sessions	Tokens (JWT)
Where state lives	Server memory/store; client holds a session-id cookie	Inside the signed token on the client; server stores nothing
Scaling	Needs sticky sessions or shared store (Redis)	Any API instance can verify — ideal for multiple servers, mobile, microservices
Revocation	Easy — delete the session	Hard — valid until expiry (hence short lifetimes + refresh tokens)
Typical use	Classic server-rendered web apps	SPAs (React) + REST APIs — our stack

Server-Side Sessions: Sticky Sessions (Session Affinity)

How Session Affinity Works

HTTP is stateless. When scaling horizontally across multiple servers, traditional session state lives in a single server's local RAM. If a client's requests are routed randomly, their session is lost.

- **Affinity Cookie:** The load balancer inspects the response to inject a unique cookie (or tracks the session cookie). On later requests, it routes the client back to the exact same server.
- **The Vulnerability:** If **Server 1** goes down, all sessions stored in its RAM are permanently lost, forcing users to re-login.
- **The Solution:** Use a centralized store like Redis instead of local RAM so any server can verify any session.



Shopping Cart Example: A user adds items to their cart on Server 1 (stored in its local RAM). If a subsequent request is randomly routed to Server 2, their cart will suddenly appear empty because Server 2 has no record of that session.

JSON Web Token (JWT) — Structure

A JSON Web Token, or **JWT**, contains three parts: the **header, payload, and signature.**

HEADER

```
eyJhbGciOiJIUzI1NiIs...
```

```
{ "alg": "HS256",
  "typ": "JWT" }
```

The **header** identifies the token *type* and *signing* algorithm.

PAYLOAD

```
eyJzdWIiOiI0MiIsInJv...
```

```
{ "sub": "42",
  "role": "Customer",
  "exp": 1718200000 }
```

The **payload** contains claims such as the user ID, role, and expiry time. This information is only **Base64-encoded**, not encrypted, so anyone can read it.

SIGNATURE

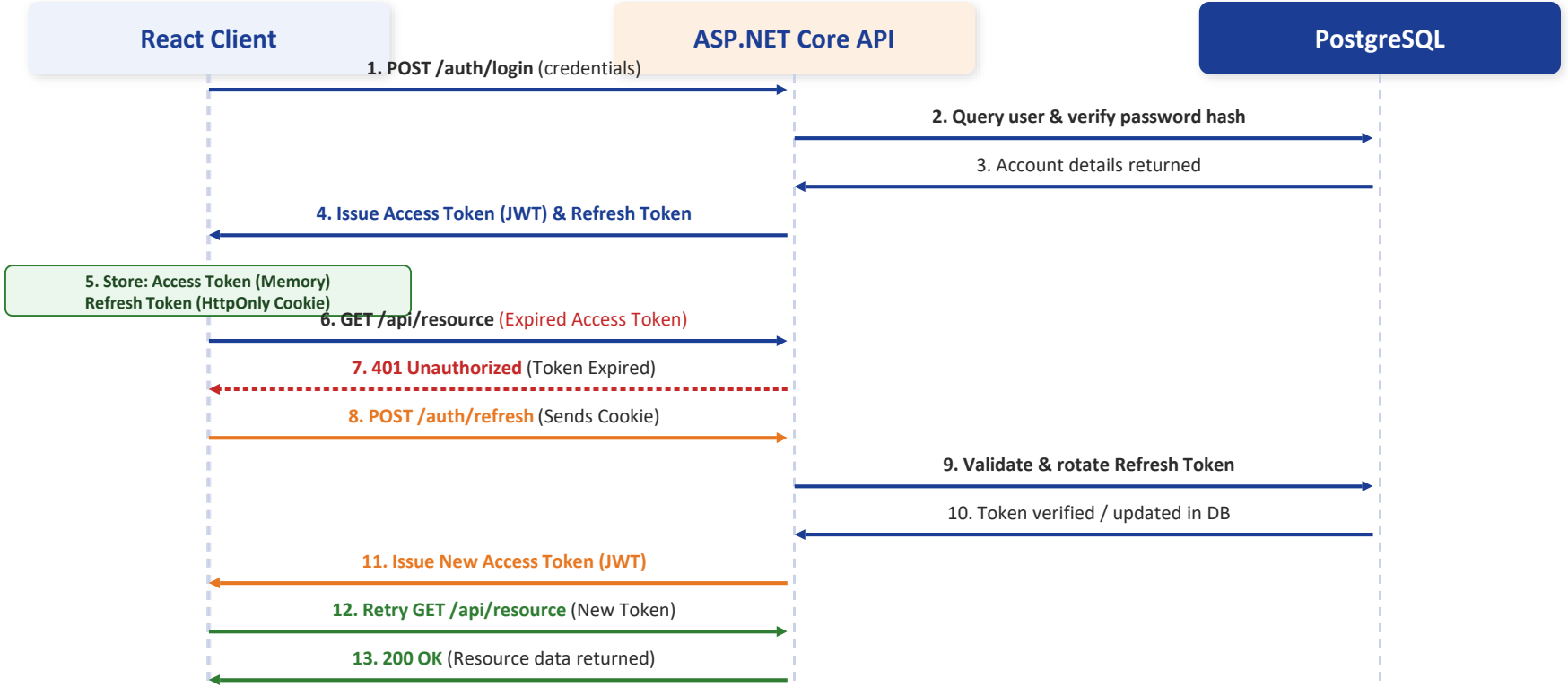
```
SflKxwRJSMeKKF2QT4f...
```

```
HMACSHA256(
  base64(header) + "." +
  base64(payload), SECRET)
```

The **signature** allows the server to verify that the token is authentic and has not been modified.

Golden rule: Never store **passwords or sensitive information** inside a JWT payload

JWT Authentication Flow



Access Tokens, Refresh Tokens & Secure Storage

	Access token	Refresh token
Purpose	Sent with every API request to prove identity	Used only to obtain a new access token
Lifetime	Short: 15–60 minutes	Long: days–weeks
Server state	None (stateless JWT)	Stored hashed in DB → can be revoked (logout, ban, rotation)
If stolen	Damage window limited by expiry	Serious — hence rotation & revocation list

Where should the React app keep tokens?

- localStorage — convenient but readable by ANY JavaScript → exposed if an XSS attack succeeds
- Recommended: refresh token in an HttpOnly, Secure, SameSite cookie (JS cannot read it); access token in memory only
- Whatever you choose: HTTPS always, short expiry, and never log tokens

Common JWT Security Risks & Mitigations

Risk	What goes wrong	Mitigation
alg: none / weak alg	Attacker strips or downgrades the signature algorithm	Server pins accepted algorithms; reject 'none'; use vetted libraries
Weak / leaked secret	Short secrets brute-forced offline → forged tokens	Long random key (256-bit+), env vars, rotation plan
No expiry validation	Stolen tokens work forever	Always set & validate exp; keep lifetimes short
Sensitive data in payload	Payload is Base64 — readable by anyone	Only ids, roles, expiry — never passwords/PII
Token theft via XSS	Injected script reads localStorage tokens	HttpOnly cookies, CSP, input/output sanitization
Trusting client claims	Role checked in React only — attacker calls API directly	Authorization enforced server-side on EVERY endpoint

The client is enemy territory: hiding a button in React is UX, not security. Every rule must also exist in the API.

Checkpoint 02 – Discussion

Discuss in pairs — 5 minutes, then we debate

D1 Your API returns 401 for an expired token AND for a wrong password. A teammate says both should be 403. Who is right, and why does it matter?

D2 A friend proposes: "Make tokens valid for 30 days so users never get logged out." Argue for and against. What design gets both convenience and safety?

D3 You can read your lecturer's JWT payload with a Base64 decoder. Is that a vulnerability? Justify using the signature mechanism.

D4 What is the WORST place mentioned today to store: (a) the JWT signing key, (b) user passwords, (c) the DB connection string?

PART 5

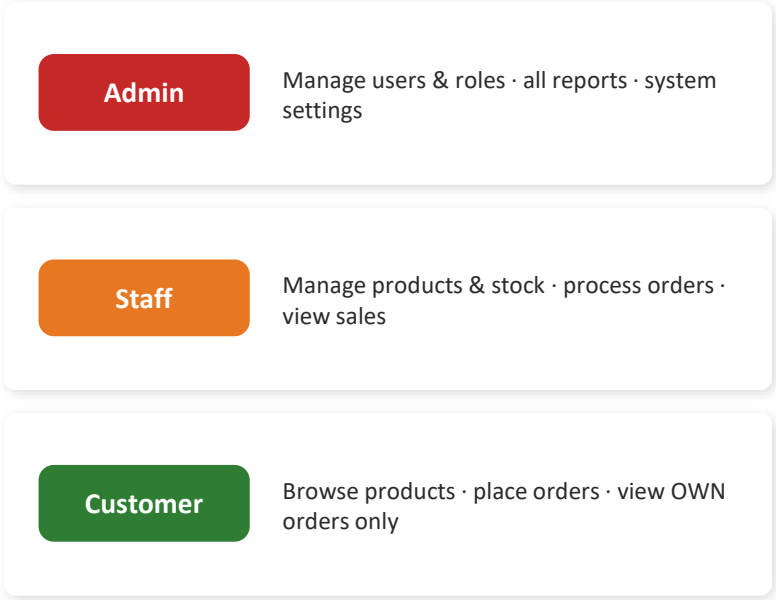
Authorization, Secure Integration & Practice

- Role-based access control & schema
- Protecting endpoints
- SQL injection & validation
- Safe errors • case study • activity • quiz



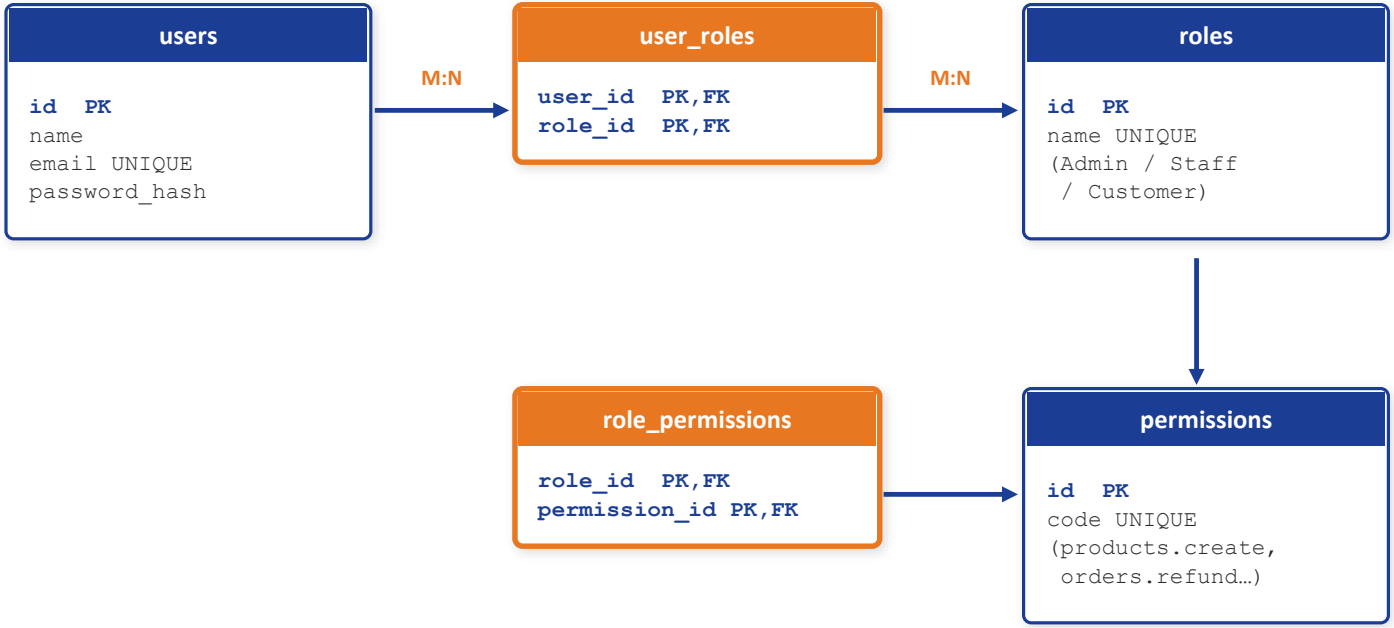
Role-Based Access Control (RBAC)

- **Permissions attach to roles, not to individual users**
  - Users gain permissions by being assigned roles
  - Manage 3 roles instead of 3,000 users' individual rights
- **Principle of least privilege: each role gets the minimum it needs**
- **It is easier to manage because a new employee can simply be assigned the appropriate role.**
- **For more control, roles can contain specific permissions such as products.create or orders.refund.**



Note "own orders only": Roles alone cannot enforce **"own orders only."** The API must compare the order's **customer ID** with the **authenticated user's ID in the JWT.**

Mapping RBAC to PostgreSQL Tables



Same pattern twice!

users↔roles and roles↔permissions are both M:N — resolved with junction tables, exactly like order_items.

At login: load the user's roles → embed as claims in the JWT → no DB lookup on later requests (until refresh).

Protecting API Endpoints by Role — ASP.NET Core

ProductsController.cs

```
[ApiController]
[Route("api/products")]
public class ProductsController : ControllerBase {

  [HttpGet]                      // public route
  [AllowAnonymous]
  public Task<IEnumerable<ProductDto>> GetAll() ...

  [HttpPost]                     // Staff or Admin
  [Authorize(Roles = "Staff,Admin")]
  public Task<IActionResult> Create(CreateProductDto d) ...

  [HttpDelete("{id}")]           // Admin only
  [Authorize(Roles = "Admin")]
  public Task<IActionResult> Delete(long id) ...
}
```

Request pipeline

1 JWT middleware: signature & expiry valid? → else 401

2 [Authorize]: authenticated? → else 401

3 Role/policy check passed? → else 403

4 Object-level check in service: owns this resource?

5 Controller action runs

Policies > raw roles for complex rules: AddPolicy("CanRefund", p => p.RequireClaim("perm", "orders.refund"))

Secure Request Flow — Frontend → Backend → Database

React (browser)

- HTTPS only
- Bearer token via interceptor
- Client-side validation (UX)
- Holds no secrets, no SQL

ASP.NET Core API

- CORS allow-list (your origin only)
- JWT validation → roles + ownership
- Server-side validation & sanitization
- Parameterized queries via EF Core

PostgreSQL

- SSL connection, private network
- Least-privilege app_user
- Constraints as last line of defence
- Only hashed passwords stored

Defence in depth: every layer assumes the previous one failed. Browser checks can be bypassed; the API re-validates; the database constrains anyway.

Validation, Sanitization & Preventing SQL Injection

- **Validate on BOTH sides — for different reasons**
  - Frontend validation provides quick feedback but can be **bypassed**.
  - Backend validation is the real security layer; check the type, format, range, and business rules.

- **ASP.NET Core supports validation through data annotations and FluentValidation.**
  - [Required], [EmailAddress], [Range(0, 1000000)]

- **React normally encodes displayed content, helping to prevent XSS attacks.**

- **SQL injection occurs when untrusted input is joined directly with an SQL command.**
  - Never build SQL queries using **string concatenation**.
  - Use **parameterized** queries or **EF Core LINQ queries**.

SQL injection

```
-- VULNERABLE: string concatenation
var sql = "SELECT * FROM users"
  + " WHERE email='" + input + "'";
-- input:  ' OR '1'='1   → returns ALL users

-- SAFE: parameterized raw SQL
db.Users.FromSqlInterpolated(
  $"SELECT * FROM users WHERE email={input}");

-- SAFE: LINQ (parameterized by EF Core)
db.Users.Where(u => u.Email == input);
```

Parameters travel SEPARATELY from the SQL text — the database can never mistake data for commands. Hand-escaping is not a substitute.

Error Handling Without Leaking Secrets

WRONG — what the client should never see

```
HTTP 500
Npgsql.PostgresException (0x80004005):
23505: duplicate key violates unique
constraint "users_email_key"
at LankaMart.Repos.UserRepo.Insert()
in C:\src\api\UserRepo.cs:line 42
```

Leaks table & constraint names, file paths, stack frames, driver versions — free reconnaissance for attackers.

RIGHT — safe, structured error response

```
HTTP 409 Conflict
{
  "type":  ".../errors/email-taken",
  "title": "Email already registered",
  "status": 409,
  "traceId": "00-8f3a...-01"
}
```

Problem Details format (RFC 9457) — built into ASP.NET Core. traceId links a user report to the full server-side log entry.

The pattern

- Global exception middleware: log full detail server-side → return generic Problem Details to the client
- Map known cases to proper codes: validation → 400 · auth → 401/403 · not found → 404 · conflict → 409 · unknown → 500

Case Study: LankaMart — Putting It All Together

Requirements → design decisions, with justification (LO4)

Requirement	Design decision	Justification
Customers register & log in	users + bcrypt hash; JWT access (15 min) + rotating refresh token	Stateless scaling for the SPA; revocation via refresh store
Orders contain many products	orders + order_items junction, composite PK, snapshot unit_price	M:N resolved; invoices immune to later price changes
Staff manage catalogue, admins manage staff	roles + user_roles; [Authorize(Roles=…)] on endpoints	Least privilege; policy change = edit role, not 3,000 users
Customers see only their own orders	Ownership check in service: order.customer_id == token sub	Role checks alone can't prevent object-level access (OWASP #1)
Stock never negative; emails unique	CHECK (stock_qty >= 0); UNIQUE(email)	Constraints enforce truth even past application bugs

Same method applies to your assignment domain: property management, student management, clinic booking …

Group Activity: Schema & Auth Flow (20 min)

Groups of 3–4 · paper or laptop · present in 60 seconds

Scenario: University Library System

Students borrow books; librarians manage the catalogue and loans; admins manage user accounts. A student may borrow up to 5 books; a book can have many copies.

- **Task 1 — ER diagram: entities, keys, cardinalities (resolve any M:N)**
- **Task 2 — RBAC: roles table + 2 protected endpoints with required roles**
- **Task 3 — Auth flow: sketch login → JWT → borrow-a-book request**
- **Task 4 — One constraint that enforces a business rule in PostgreSQL**

Marking lens (same as the assignment)

- Junction tables where needed?
- FKs on the correct side?
- 3NF — or justified exceptions?
- 401 vs 403 used correctly?
- Object-level check identified? (a student sees only own loans)

Quick Quiz — 5 Questions

1 Which normal form is violated when module_name sits in a table keyed by (student_id, module_id)?

A) 1NF B) 2NF C) 3NF

2 An order_items table linking orders and products is an example of resolving a …

A) 1:1 B) 1:N C) M:N relationship

3 A valid but expired JWT arrives at a protected endpoint. The correct response is …

A) 400 B) 401 C) 403

4 Which part of a JWT prevents tampering?

A) Header B) Payload C) Signature

5 The single most effective defence against SQL injection is …

A) Input length limits B) Parameterized queries C) Hiding error messages

Lecture Summary — Key Takeaways

Design the data first	3NF schemas with proper keys, junction tables for M:N, constraints and FK indexes — in PostgreSQL, with migrations.
Integrate through layers	Controller → service → repository → EF Core/Npgsql; env vars for connection strings; pooled connections; typed error handling.
Authenticate with JWT	Stateless signed tokens: short-lived access + revocable rotating refresh; bcrypt-hashed passwords; secure token storage.
Authorize everywhere	RBAC via roles/junction tables, [Authorize] policies on every endpoint, PLUS object-level ownership checks.
Defend in depth	Validate both sides, parameterized queries only, safe Problem-Details errors, least-privilege DB users, secrets in env config.

Learning Outcomes, Next Lecture & Preparation

LO	Covered today by…
LO2	Designing PostgreSQL schemas and integrating them with ASP.NET Core (EF Core + Npgsql) to build full-stack applications
LO3	Best practices for secure integration: layered architecture, env configuration, validation, parameterized queries, safe error handling
LO4	Selecting and justifying PostgreSQL, JWT-based authentication and RBAC against project requirements (case study & activity)



Next: Agentic Software Development (1 of 3)

AI agents in engineering workflows, AI-assisted design and coding — you will use today's schema and auth design as the playground for AI-assisted implementation.



Before the lab / next week

- Install PostgreSQL 16 + pgAdmin (or Docker image)
- Sketch the ER diagram for YOUR assignment domain
- Read: EF Core 'Getting Started with PostgreSQL' (Npgsql docs)
- Skim OWASP Top 10 — A01 & A07

References & Further Reading

- **Module-recommended texts**
  - Freeman, E. & Robson, E. (2020) Head First Design Patterns, 2nd edn. O'Reilly Media.
  - Ford, N., Richards, M., Sadalage, P. & Dehghani, Z. (2021) Software Architecture: The Hard Parts. O'Reilly Media.
  - Biswas, A. & Talukdar, W. (2025) Building Agentic AI Systems. Packt Publishing.
- **Documentation (primary sources)**
  - PostgreSQL 16 Documentation — postgresql.org/docs (DDL, constraints, indexes)
  - Npgsql & EF Core PostgreSQL provider — npgsql.org
  - Microsoft Learn — ASP.NET Core Security: Authentication & Authorization
- **Security references**
  - OWASP Top 10 (2021) & OWASP API Security Top 10 (2023) — owasp.org
  - RFC 7519 (JSON Web Token) · RFC 9457 (Problem Details for HTTP APIs)
  - OWASP Cheat Sheet Series: SQL Injection Prevention · Password Storage